

ASSESSMENT TOOL - 1 : PSEUDOCODE WRITING TEST

Course Code	ECA1407
Course Name	Embedded Systems
Course Outcome	CO3
Assessment Tool	Pseudocode Writing Test
Question 1 No.10	Write pseudocode for resource allocation in an industrial plant while avoiding deadlock and starvation.
Question 2 No.11	Write pseudocode for sending sensor data to a logging task using a message queue.
Question 3 No.12	Write pseudocode for communication between two tasks using a mailbox.
Question 4 No.13	Write pseudocode showing blocking and non-blocking queue operations in RTOS
Question 5 No.14	Write pseudocode for a sensor task writing data to a pipe and a consumer task reading it.
Maximum Marks	50 (5 × 10)
Bloom's Taxonomy Level	BL4

Evaluation Rubric (10 Marks per Question)

Criteria	Max Marks	Q1	Q2	Q3	Q4	Q5
Understanding of Problem & RTOS Concept	2	2	2	2	2	2
Algorithm Design	3	3	3	3	3	3
Use of Control Structures	2	2	2	2	2	2
Logical Flow & Structure of Pseudocode	1	1	1	1	1	1
Completeness	2	2	2	2	2	2
Total Marks	10	10 /10	10 /10	10 /10	10 /10	10 /10

Code of Conduct:

I K.Manoj Kumar (192472395) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:



QUESTION NO: 10 Write pseudocode for resource allocation in an industrial plant while avoiding deadlock and starvation.

1. PROBLEM UNDERSTANDING

In an industrial plant, multiple tasks request shared resources such as robotic arms, welding machines, conveyors and cameras. If tasks hold some resources and wait for others held by different tasks, deadlock can occur. Low priority tasks may wait forever if higher priority tasks keep getting resources. This is called starvation.

RTOS MECHANISMS USED

- Mutex – protects resource allocation
- Fixed resource ordering – prevents deadlock
- Waiting queue – stores waiting tasks
- Priority aging – prevents starvation

RESOURCES IN THE PLANT (USED ORDER)

1. Robot Arm
 2. Welding Machine
 3. Conveyor
 4. Inspection Camera
- (All tasks must request in this order)

2. REAL-TIME INDUSTRIAL EXAMPLE

Scenario: Automated Automobile Manufacturing Plant



Three tasks require different combinations of resources.

Task	Required Resources	Priority
Task 1 – Body Welding	Robot Arm + Welding Machine	1
Task 2 – Inspection	Conveyor + Camera	2
Task 3 – Robotic Inspection	Robot Arm + Camera	3

Without proper allocation, deadlock and starvation may occur.

3. PSEUDOCODE

3.1 SYSTEM INITIALIZATION

```
BEGIN
  CREATE MUTEX ResourceLock

  CREATE RESOURCE RobotArm
  CREATE RESOURCE WeldingMachine
  CREATE RESOURCE Conveyor
  CREATE RESOURCE InspectionCamera

  CREATE QUEUE WaitingQueue

  CREATE TASK Task1 WITH PRIORITY 1
  CREATE TASK Task2 WITH PRIORITY 2
  CREATE TASK Task3 WITH PRIORITY 3

  START RTOS SCHEDULER
END
```

3.2 INDUSTRIAL TASK

```
TASK IndustrialTask(task_id, required_resources)
BEGIN
  SORT required_resources IN ASCENDING ORDER

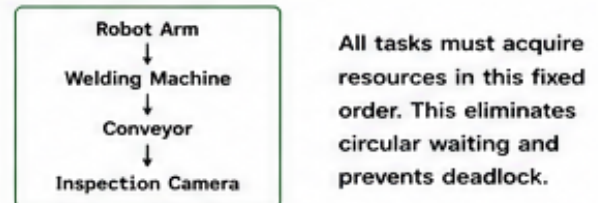
  WHILE resources are NOT available DO
    INCREMENT wait_time[task_id]
    ENQUEUE WaitingQueue, task_id
    IF wait_time[task_id] > MAX_WAIT_TIME THEN
      INCREASE PRIORITY OF task_id // Priority aging
    END IF
    WAIT until resources become available
    DEQUEUE WaitingQueue, task_id
  END WHILE

  WAIT(ResourceLock)
  FOR EACH res IN required_resources DO
    ALLOCATE res TO task_id
  END FOR
  SIGNAL(ResourceLock)
  PERFORM industrial_operation() // Critical section
  WAIT(ResourceLock)
  FOR EACH res IN required_resources DO
    RELEASE res
  END FOR
  SIGNAL waiting tasks
  SIGNAL(ResourceLock)
END TASK
```

3.3 RESOURCE MANAGER TASK

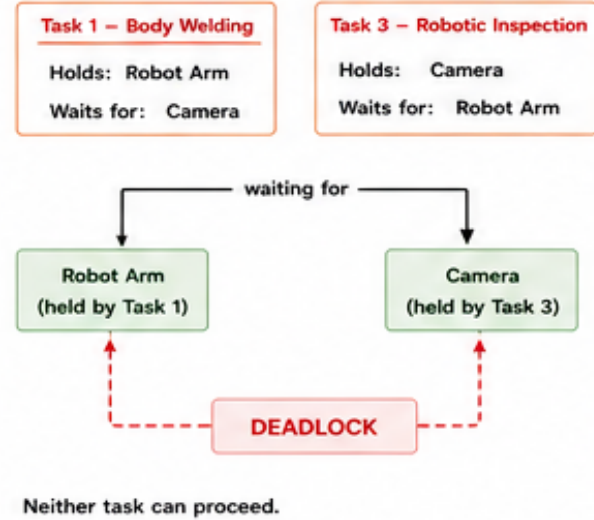
```
TASK ResourceManager
BEGIN
  WHILE TRUE DO
    FOR EACH task IN WaitingQueue DO
      IF all required resources available THEN
        IF wait_time[task] > MAX_WAIT_TIME THEN
          INCREASE PRIORITY OF task // Aging
        END IF
        ALLOCATE resources TO task
        DEQUEUE WaitingQueue, task
      END IF
    END FOR
    WAIT 10 ms
  END WHILE
END TASK
```

RESOURCE ACQUISITION ORDER (DEADLOCK PREVENTION)

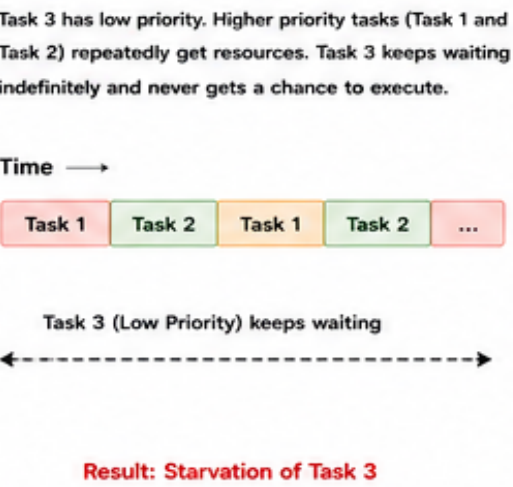


4. DEADLOCK AND STARVATION EXAMPLE

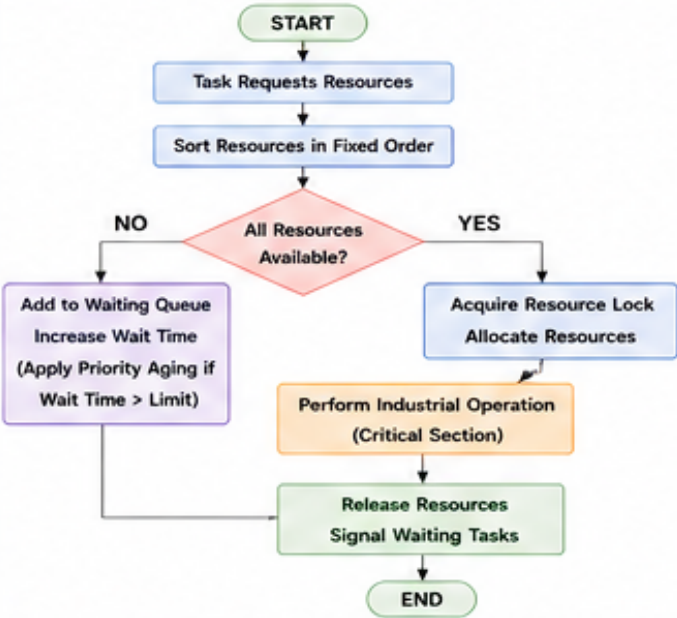
4.1 DEADLOCK SCENARIO (Without Proper Allocation)



4.2 STARVATION SCENARIO (Without Priority Aging)



5. LOGICAL FLOW DIAGRAM



6. HOW THIS SOLVES THE PROBLEM

Deadlock Prevention

- ✓ All tasks request resources in the same fixed order.
- ✓ No circular waiting condition.
- ✓ Only one task can hold a resource at a time (Mutex).
- ✓ Deadlock cannot occur.

Starvation Prevention

- ✓ Each task's waiting time is monitored.
- ✓ If waiting time exceeds the limit, task priority is increased.
- ✓ Low priority tasks eventually get resources.
- ✓ Starvation is eliminated.

RESOURCE OPERATIONS SUMMARY

Operation	Description	RTOS Mechanism
Request Resources	Task requests required resources	Check availability, queue
Allocate Resources	Resources are allocated to task	Mutex + Fixed Order
Use Resources	Task performs industrial operation	Critical Section
Release Resources	Resources returned after use	Mutex
Wake Waiting Tasks	Notify waiting tasks	Signal / Queue

7. KEY POINTS

- Fixed resource order prevents deadlock.
- Mutex ensures mutual exclusion during allocation.
- Waiting queue stores tasks safely.
- Priority aging increases priority of waiting tasks.
- All tasks eventually get resources (no starvation).
- Resources are released after operation.
- Improves reliability and real-time performance of the plant.

8. ASSUMPTIONS

- Each resource can be used by only one task at a time.
- All tasks follow the same resource acquisition order.
- RTOS supports mutexes, queues and priorities.
- MAX_WAIT_TIME and priorities are pre-defined.
- Tasks release resources after completing operation.

9. REAL-TIME BENEFITS IN INDUSTRY

This approach prevents machine stoppages, deadlocks and delays in production lines. It ensures all industrial tasks get equal opportunity, improves safety, increases throughput and maintains smooth operation of the plant with real-time constraints.

10. GLOSSARY

- Resource** : Shared item used by tasks (Robot Arm, Camera, etc.)
- Mutex** : Binary semaphore for mutual exclusion (0 or 1)
- Waiting Queue** : Holds tasks waiting for resources
- Priority Aging** : Increasing priority of waiting task to avoid starvation
- Deadlock** : Two or more tasks waiting forever for resources
- Starvation** : Indefinite waiting due to lower priority

QUESTION NO: 11 Write pseudocode for sending sensor data to a logging task using a message queue.

1. PROBLEM UNDERSTANDING

In a real-time embedded system, a sensor continuously collects data (e.g., temperature, pressure, vibration). The data must be stored or logged for monitoring and analysis. If the sensor task directly writes to a storage device (SD card / Flash / Cloud), it may get delayed due to slow I/O operations, causing data loss.

Therefore, the sensor task sends the data to a logging task using a message queue. The logging task receives the data from the queue and logs it safely.

RTOS MECHANISM USED

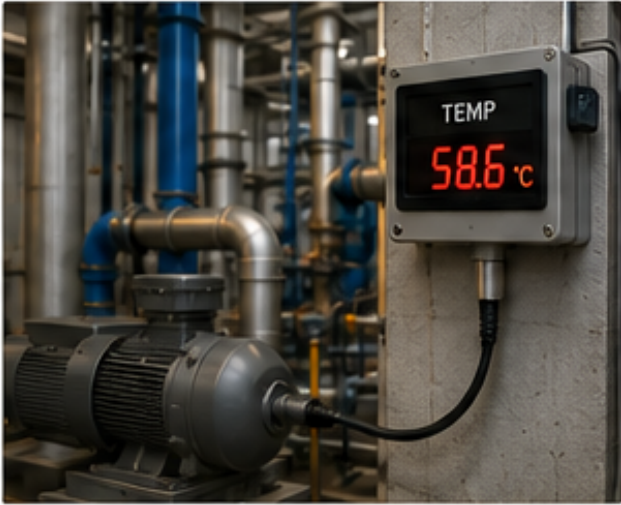
Message Queue
(Producer-Consumer Communication)

DATA STRUCTURES (GLOBAL DEFINITIONS)

```
DEFINE QUEUE SensorQueue
DEFINE CONSTANT QUEUE_SIZE = 10
DATA STRUCTURE: SensorData
- timeStamp : TIME
- sensorId : INTEGER
- value : FLOAT
- status : INTEGER
```

REAL-TIME INDUSTRIAL EXAMPLE

Scenario: Industrial Temperature Monitoring System



A temperature sensor mounted on an industrial machine continuously measures the temperature. The readings are not sent directly to the storage (which is slow). Instead, Sensor Task sends the data to Logging Task through a Message Queue. The Logging Task stores the data in SD card / Flash / Database.

2. PSEUDOCODE

SYSTEM INITIALIZATION

```
BEGIN
  CREATE QUEUE SensorQueue
  WITH SIZE = QUEUE_SIZE

  CREATE TASK SensorTask
  WITH PRIORITY 2

  CREATE TASK LoggingTask
  WITH PRIORITY 1

  START SCHEDULER

END
```

TASK: SENSOR TASK (PRODUCER)

```
TASK SensorTask
BEGIN
  WHILE TRUE DO
    DECLARE data : SensorData

    data.timeStamp = GET_SYSTEM_TIME()
    data.sensorId = 1 // Temperature Sensor
    data.value = READ_TEMPERATURE_SENSOR()
    data.status = 0 // 0 = Normal

    IF SensorQueue is NOT FULL THEN
      SEND data TO SensorQueue // Non-blocking send
    ELSE
      INCREMENT queueOverflowCount // Handle overflow
      SIGNAL overflowEvent
    END IF

    WAIT 1000 ms // Sample every 1 second
  END WHILE
END TASK
```

3. PSEUDOCODE (CONTINUED...)

TASK: LOGGING TASK (CONSUMER)

```
TASK LoggingTask
BEGIN
  WHILE TRUE DO
    DECLARE data : SensorData

    WAIT FOR message FROM SensorQueue // Blocking receive

    IF message received THEN
      WRITE_TO_SD_CARD(data.timeStamp,
        data.sensorId,
        data.value,
        data.status)

      UPDATE logCount
    END IF
  END WHILE
END TASK
```

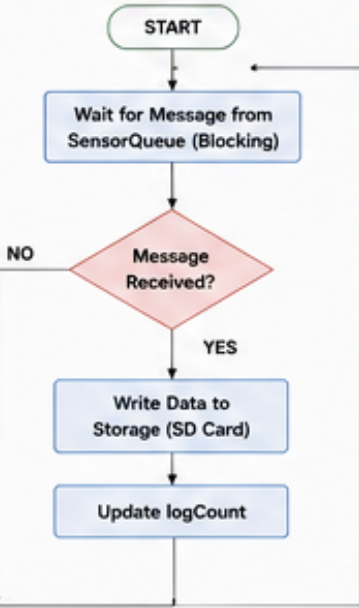
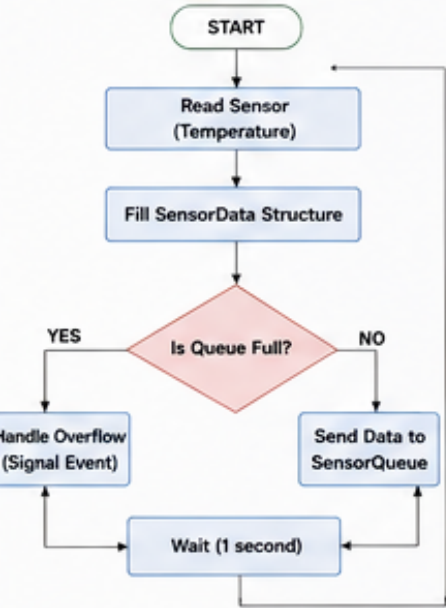
QUEUE OPERATIONS SUMMARY

Operation	Performed By	Purpose
SEND(data)	Sensor Task	Sends sensor data to queue
RECEIVE(data)	Logging Task	Receives data from queue
QUEUE FULL?	Sensor Task	Check before sending
WAIT (blocking)	Logging Task	Wait until data is available
SIGNAL(overflow)	Sensor Task	Notify overflow condition

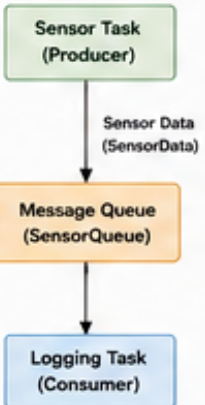
4. LOGICAL FLOW DIAGRAM

SENSOR TASK FLOW (PRODUCER)

LOGGING TASK FLOW (CONSUMER)



MESSAGE FLOW



5. HOW THIS PSEUDOCODE SOLVES THE PROBLEM

- ✓ Decouples Sensor & Logging Tasks: Sensor task is not blocked by slow storage operations.
- ✓ Prevents Data Loss: Queue acts as a buffer between producer and consumer.
- ✓ Improves Real-time Performance: Sensor task completes quickly and maintains sampling rate.
- ✓ Handles Overflows: If queue is full, overflow event is signaled for further action.
- ✓ Reliable Logging: Logging task writes data safely to storage without affecting sensor sampling.

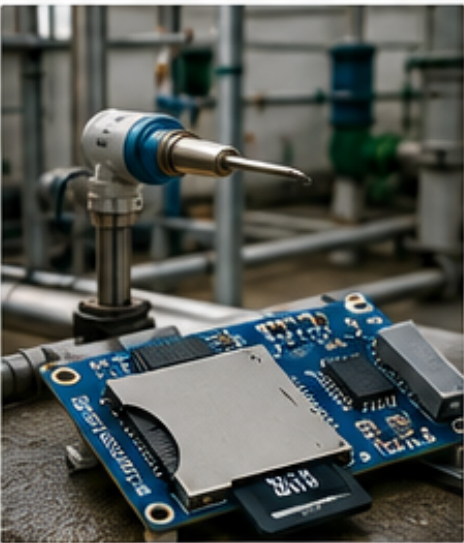
PROBLEM WITHOUT QUEUE



REAL-TIME BENEFIT IN INDUSTRY

In an industrial temperature monitoring system, sensors must provide readings every second. Using a message queue ensures that even if the SD card writing takes longer time, no temperature reading is lost and the system remains reliable and responsive.

6. REAL-WORLD IMPLEMENTATION EXAMPLE



Temperature sensor (e.g., RTD/PT100) mounted on the machine sends data to the microcontroller.

Microcontroller (RTOS) -> Sensor Task places data into Message Queue.

Logging Task writes data to SD Card for future analysis.

RESULT:

Continuous temperature monitoring with zero data loss and reliable logging.

7. KEY POINTS

- Message Queue provides safe task-to-task communication.
- Sensor task (Producer) sends data; Logging task (Consumer) receives it.
- Queue decouples fast sensor sampling from slow storage operations.
- Blocking receive improves CPU efficiency.
- System remains real-time and reliable.

8. ASSUMPTIONS

- Queue size is sufficient to handle temporary bursts.
- SD card writing is slower than sensor sampling.
- RTOS supports message queues and task scheduling.
- Sensor provides valid readings at every sampling interval.
- Overflow handling mechanism is implemented.

9. GLOSSARY

- Message Queue** : RTOS object used for task-to-task communication.
- Producer** : Task that sends data to the queue.
- Consumer** : Task that receives data from the queue.
- Blocking Receive** : Task waits until data is available in the queue.
- Overflow** : Condition when queue is full and cannot accept new data.

QUESTION NO: 12 Write pseudocode for communication between two tasks using a mailbox.

1. PROBLEM UNDERSTANDING

In an RTOS, two tasks often need to exchange data safely. One task produces information and the other task consumes it. Using a shared variable may cause race conditions and data corruption. A Mailbox allows one task to send a single message to another task. The receiver reads the latest message and the mailbox holds only one message at a time.

RTOS MECHANISM USED

Mailbox
(One-to-One Communication)

DATA STRUCTURES (GLOBAL DEFINITIONS)

DEFINE MAILBOX TempMailbox
DATA STRUCTURE: TemperatureMsg

- temperature : FLOAT
- timeStamp : TIME
- status : INTEGER

CONSTANT MAILBOX_SIZE = 1 // Mailbox holds only single message

REAL-TIME INDUSTRIAL EXAMPLE

Scenario: Industrial Motor Temperature Monitoring and Fan Control



A temperature sensor mounted on the motor reads the temperature periodically. The temperature data is sent to the Fan Control Task using a Mailbox. The Fan Control Task uses this data to decide whether to turn ON or OFF the cooling fan.

2. PSEUDOCODE

SYSTEM INITIALIZATION

```
BEGIN
CREATE MAILBOX TempMailbox
WITH SIZE = MAILBOX_SIZE

CREATE TASK TemperatureTask
WITH PRIORITY 2

CREATE TASK FanControlTask
WITH PRIORITY 1

START SCHEDULER

END
```

TASK: TEMPERATURE TASK (SENDER)

```
TASK TemperatureTask
BEGIN
DECLARE msg : TemperatureMsg

WHILE TRUE DO
msg.temperature = READ_TEMP_SENSOR()
msg.timeStamp = GET_SYSTEM_TIME()
msg.status = 0 // 0 = Normal, 1 = Fault

SEND msg TO TempMailbox // Blocking send
// If mailbox is full, sender waits until
// receiver reads the message

WAIT 1000 ms // Sample every 1 second
END WHILE
END TASK
```

TASK: FAN CONTROL TASK (RECEIVER)

```
TASK FanControlTask
BEGIN
DECLARE msg : TemperatureMsg

WHILE TRUE DO
RECEIVE msg FROM TempMailbox // Blocking receive

IF msg.temperature > 80.0 THEN
TURN_FAN_ON()
ELSE IF msg.temperature < 60.0 THEN
TURN_FAN_OFF()
END IF

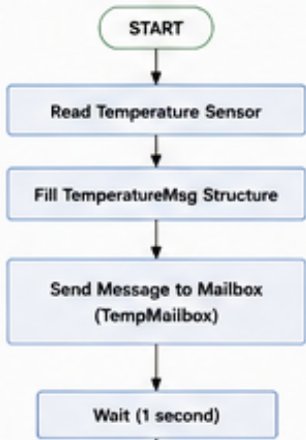
LOG_TEMPERATURE(msg.temperature, msg.timeStamp)
END WHILE
END TASK
```

MAILBOX OPERATIONS SUMMARY

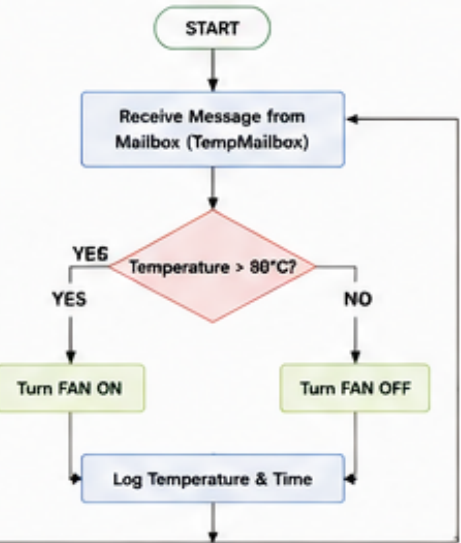
Operation	Performed By	Purpose	Behaviour
SEND(message)	Temperature Task	Sends temperature data to mailbox	Blocks if mailbox is full
RECEIVE(message)	Fan Control Task	Receives latest message from mailbox	Blocks if mailbox is empty
MAILBOX (SIZE = 1)	System	Stores only the latest message	New message overwrites old if not read (depending on RTOS)

3. LOGICAL FLOW DIAGRAM

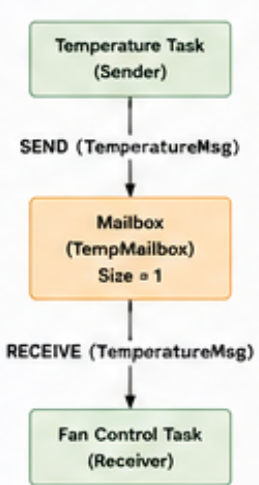
TEMPERATURE TASK FLOW (SENDER)



FAN CONTROL TASK FLOW (RECEIVER)



MAILBOX COMMUNICATION



4. HOW THIS PSEUDOCODE SOLVES THE PROBLEM

PROBLEM WITHOUT MAILBOX

- ❌ Shared variable between tasks may lead to race conditions and data corruption.
- ❌ Receiver task may read data while sender is updating it.
- ❌ Needs continuous polling to check for new data.
- ❌ Wasted CPU time and increased latency.
- ❌ Old data may be read if not synchronized properly.

SOLUTION USING MAILBOX

- ✅ Mailbox provides safe and synchronized communication.
- ✅ Sender writes data and receiver reads it atomically.
- ✅ Receiver task blocks until new data arrives.
- ✅ No polling, saves CPU time.
- ✅ Always receives the latest valid data.

REAL-TIME BENEFIT IN INDUSTRY

In industrial motor systems, accurate and timely temperature data is critical. Using a mailbox ensures the fan control task always gets the latest temperature reading without data loss or corruption. This prevents overheating and improves equipment reliability and safety.

5. REAL-WORLD IMPLEMENTATION EXAMPLE



Temperature Sensor
(PT100 / Thermocouple)
on Motor

Fan Control Panel
(Controlled by RTOS Task
using Mailbox Data)

6. KEY POINTS

- Mailbox is used for one-to-one task communication.
- Mailbox can hold only one message at a time (SIZE = 1).
- Sender blocks if mailbox is full.
- Receiver blocks if mailbox is empty.
- Ensures data consistency and prevents race conditions.
- Improves system efficiency and real-time performance.

7. ASSUMPTIONS

- RTOS supports mailbox API.
- Only one sender and one receiver for this mailbox.
- Temperature sensor reading is available.
- Mailbox size is sufficient (SIZE = 1 for latest data).
- Fan control depends on temperature thresholds.

8. GLOSSARY

- Mailbox** : RTOS object used to send messages between two tasks.
- Sender** : Task that sends the message.
- Receiver** : Task that receives the message.
- Blocking Send** : Sender waits if mailbox is full.
- Blocking Receive** : Receiver waits if mailbox is empty.
- One-to-One Communication** : Communication between exactly two tasks.

QUESTION NO: 13 Write pseudocode showing blocking and non-blocking queue operations in RTOS.

1. PROBLEM UNDERSTANDING

In an RTOS, tasks communicate using a queue. When a task tries to receive data from an empty queue, it can either wait (blocking) until data arrives, or return immediately (non-blocking) and continue other work.

This question demonstrates both blocking and non-blocking queue operations with one producer task and two consumer tasks.

RTOS MECHANISM USED

Message Queue
(Blocking Receive & Non-Blocking Receive)

DATA STRUCTURES (GLOBAL DEFINITIONS)

```
DEFINE QUEUE DataQueue
DEFINE CONSTANT QUEUE_SIZE = 10
```

DATA STRUCTURE: SensorData

- value : INTEGER
- timeStamp : TIME

REAL-TIME INDUSTRIAL EXAMPLE

Scenario: Industrial Vibration Monitoring System



A vibration sensor mounted on the machine continuously produces vibration data. One task stores it in a queue. Two consumer tasks process the data differently:

- Task using BLOCKING receive waits for data.
- Task using NON-BLOCKING receive does other work if no data is available.

2. PSEUDOCODE

```
SYSTEM INITIALIZATION
BEGIN
    CREATE QUEUE DataQueue
        WITH SIZE = QUEUE_SIZE

    CREATE TASK SensorTask
        WITH PRIORITY 2

    CREATE TASK BlockingConsumerTask
        WITH PRIORITY 1

    CREATE TASK NonBlockingConsumerTask
        WITH PRIORITY 1

    START SCHEDULER
END

TASK: SENSOR TASK (PRODUCER)
TASK SensorTask
BEGIN
    WHILE TRUE DO
        DECLARE data : SensorData

        data.value = READ_VIBRATION_SENSOR()
        data.timeStamp = GET_SYSTEM_TIME()

        IF QUEUE_SEND(DataQueue, data, NO_WAIT) == SUCCESS THEN
            // Data sent successfully
        ELSE
            // Queue full, handle overflow
            INCREMENT lostDataCount
        END IF

        WAIT 100 ms // Sample every 100 ms
    END WHILE
END TASK

TASK: BLOCKING CONSUMER
TASK BlockingConsumerTask
BEGIN
    WHILE TRUE DO
        DECLARE data : SensorData

        // BLOCKING RECEIVE
        status = QUEUE_RECEIVE(DataQueue,
                                &data, WAIT_FOREVER)
        // Task blocks here if queue is empty

        IF status == SUCCESS THEN
            PROCESS_DATA_BLOCKING(data)
        END IF
    END WHILE
END TASK

TASK: NON-BLOCKING CONSUMER
TASK NonBlockingConsumerTask
BEGIN
    WHILE TRUE DO
        DECLARE data : SensorData

        // NON-BLOCKING RECEIVE
        status = QUEUE_RECEIVE(DataQueue,
                                &data, NO_WAIT)

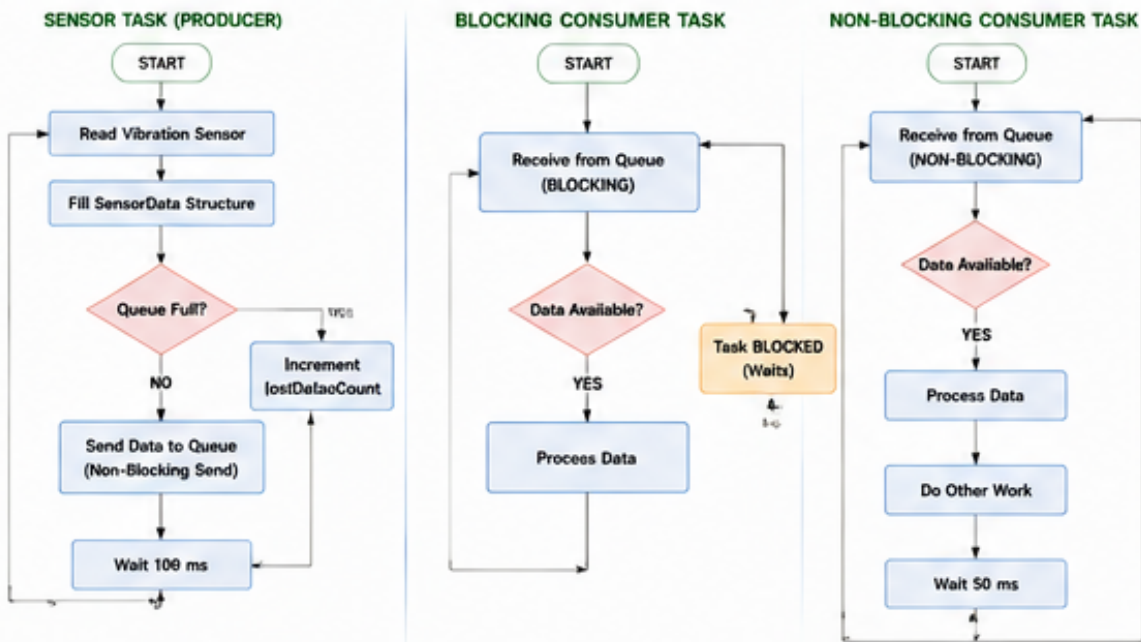
        IF status == SUCCESS THEN
            PROCESS_DATA_NONBLOCKING(data)
        ELSE
            // No data available
            PERFORM_OTHER_WORK()
        END IF

        WAIT 50 ms // Small delay before next try
    END WHILE
END TASK
```

QUEUE OPERATIONS SUMMARY

Operation	Function Used	Behavior When Queue Empty	Behavior When Queue Full	Used By
Blocking Receive	QUEUE_RECEIVE(queue, &data, WAIT_FOREVER)	Task waits (BLOCKED state) until data arrives	Not applicable	BlockingConsumerTask
Non-Blocking Receive	QUEUE_RECEIVE(queue, &data, NO_WAIT)	Returns immediately with status = FAIL	Not applicable	NonBlockingConsumerTask
Non-Blocking Send	QUEUE_SEND(queue, data, NO_WAIT)	Not applicable	Returns FAIL immediately	SensorTask

3. LOGICAL FLOW DIAGRAM



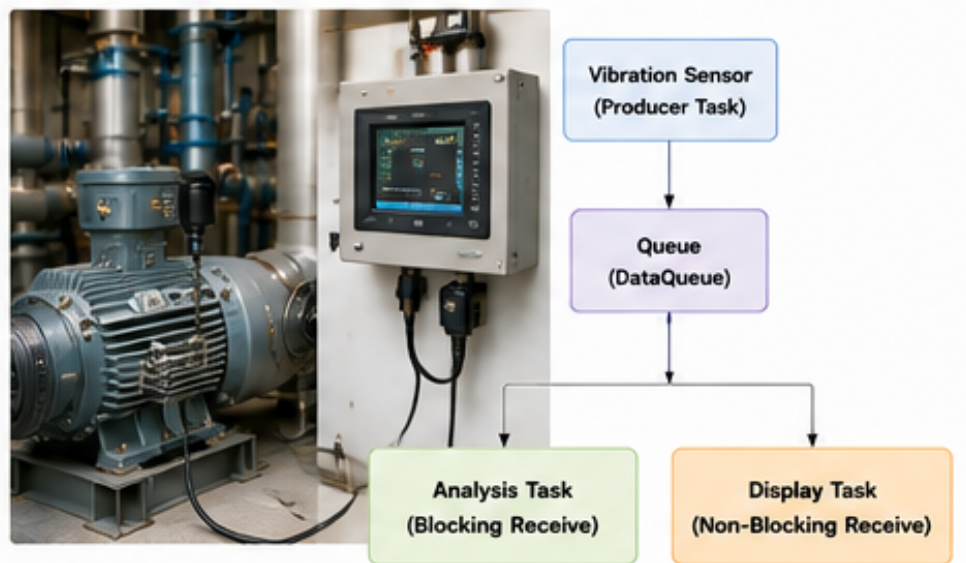
4. HOW THIS PSEUDOCODE SOLVES THE PROBLEM

- ✓ Blocking receive ensures the task waits without wasting CPU time when no data is available.
- ✓ Non-blocking receive allows the task to continue other useful work without being blocked.
- ✓ This improves CPU utilization and system responsiveness.
- ✓ Queue decouples producer and consumers, ensuring safe data transfer.
- ✓ Overflow handling prevents data loss when queue is full.
- ✓ System becomes efficient, reliable, and suitable for real-time industrial applications.

REAL-TIME BENEFIT IN INDUSTRY

In vibration monitoring, the sensor generates data continuously. Using blocking receive for critical analysis ensures no data is missed, while non-blocking receive allows background tasks (like display update, communication, or diagnostics) to run without delay. This reduces CPU wastage, prevents data loss, and improves overall system performance.

5. REAL-WORLD IMPLEMENTATION EXAMPLE



6. KEY POINTS

- Blocking receive puts the task in BLOCKED state until data arrives.
- Non-blocking receive returns immediately if queue is empty.
- Non-blocking send returns immediately if queue is full.
- Queue provides safe producer-consumer communication.
- Proper selection of blocking or non-blocking improves real-time performance.

7. ASSUMPTIONS

- Queue size is sufficient for normal operation.
- Sensor sampling rate is faster than or equal to consumer processing rate.
- RTOS supports queue API with timeout or no-wait options.
- Tasks are properly prioritized by the scheduler.
- Overflow counter is used to monitor data loss.

8. GLOSSARY

Blocking Receive	: Task waits until data is available in the queue.
Non-Blocking Receive	: Task returns immediately if no data is available.
Queue	: RTOS object used for task-to-task communication.
Producer	: Task that sends data to the queue.
Consumer	: Task that receives data from the queue.

QUESTION NO: 14 Write pseudocode for a sensor task writing data to a pipe and a consumer task reading it.

1. PROBLEM UNDERSTANDING

In an RTOS, a sensor task periodically reads data from a sensor and sends it to another task for processing or logging. A pipe provides a unidirectional communication channel following FIFO (First In First Out) order.

The sensor task (producer) writes data to the pipe and the consumer task reads the data from the pipe.

This ensures safe, efficient and synchronized data transfer without busy waiting.

RTOS MECHANISM USED

- Pipe – Unidirectional communication with FIFO behavior

PIPE CHARACTERISTICS

- Unidirectional: Data flows from writer to reader only.
- FIFO: Data is read in the same order as written.
- Bounded Buffer: Has a fixed size.
- If full: Writer blocks until space is available.
- If empty: Reader blocks until data arrives.

2. REAL-TIME INDUSTRIAL EXAMPLE

Scenario: Temperature Monitoring System in Boiler Plant



A temperature sensor in a boiler continuously measures temperature. The sensor task reads the temperature and sends it to a logging task using a pipe. The logging task reads the data and stores it in memory or sends it to a display system.

Task	Function
Sensor Task (Producer)	Reads temperature and writes data to pipe
Consumer Task	Reads data from pipe and logs/displays it
Pipe	Transfers data from sensor task to consumer task

3.1 SENSOR TASK (WRITER / PRODUCER)

```
TASK SensorTask
BEGIN
    DECLARE tempValue : INTEGER
    DECLARE status : INTEGER

    // Sensor task initialization
    INIT_TEMPERATURE_SENSOR()

    WHILE TRUE DO
        // Read temperature from sensor
        tempValue = READ_TEMPERATURE_SENSOR()

        // Write data to pipe (blocks if pipe is full)
        status = WRITE_PIPE(TempPipe, &tempValue, sizeof(tempValue))

        IF status == PIPE_OK THEN
            // Data written successfully
            SIGNAL(sensorEvent)
        ELSE
            // Handle write error (optional)
            LOG_ERROR("Pipe write failed")
        END IF

        WAIT(DELAY_SENSOR_SAMPLING) // e.g., 1000 ms
    END WHILE
END TASK
```

3.2 CONSUMER TASK (READER)

```
TASK ConsumerTask
BEGIN
    DECLARE tempValue : INTEGER
    DECLARE status : INTEGER

    WHILE TRUE DO
        // Read data from pipe (blocks if pipe is empty)
        status = READ_PIPE(TempPipe, &tempValue, sizeof(tempValue))

        IF status == PIPE_OK THEN
            // Process the received data
            LOG_TEMPERATURE(tempValue)
            UPDATE_DISPLAY(tempValue)
            STORE_IN_MEMORY(tempValue)
        ELSE
            // Handle read error (optional)
            LOG_ERROR("Pipe read failed")
        END IF
    END WHILE
END TASK
```

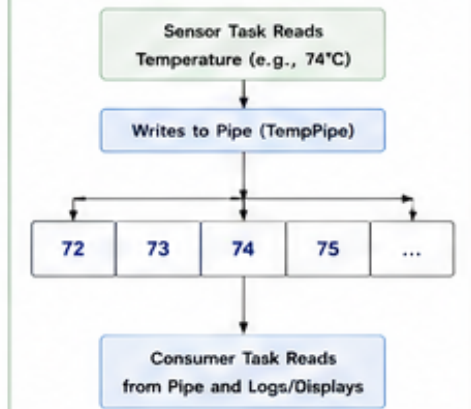
3.3 PIPE CREATION (SYSTEM INITIALIZATION)

```
BEGIN
    // Create a pipe named TempPipe with size 10 and data size of INTEGER
    CREATE_PIPE(TempPipe, PIPE_LENGTH = 10, ITEM_SIZE = sizeof(INTEGER))
    CREATE_TASK(SensorTask, PRIORITY = 2, STACK_SIZE = 1024)
    CREATE_TASK(ConsumerTask, PRIORITY = 1, STACK_SIZE = 1024)
    START_RTOS_SCHEDULER()
END
```

PIPE CONFIGURATION

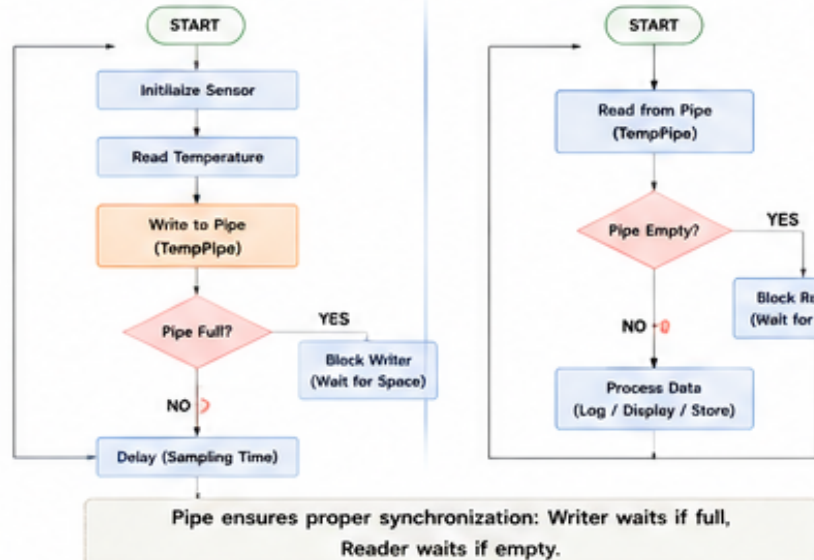
Pipe Name	: TempPipe
Length	: 10 (number of items)
Item Size	: INTEGER (e.g., 2 bytes)
Direction	: SensorTask → ConsumerTask
Behavior When Full	: Writer blocks
Behavior When Empty	: Reader blocks

EXAMPLE DATA FLOW

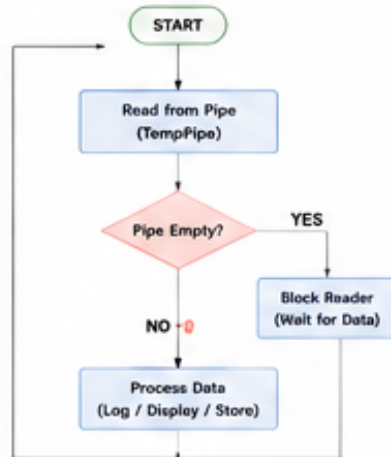


4. LOGICAL FLOW DIAGRAM

4.1 SENSOR TASK (WRITER)



4.2 CONSUMER TASK (READER)



5. HOW THIS SOLVES THE PROBLEM

- ✓ Pipe provides a safe and efficient way for the sensor task to send data to the consumer task.
- ✓ If the pipe is full, the sensor task automatically blocks until space is available.
- ✓ If the pipe is empty, the consumer task blocks until data is available.
- ✓ This prevents data loss and ensures FIFO order of data.
- ✓ CPU is not wasted in busy waiting; tasks block and the scheduler runs other tasks.
- ✓ This method is reliable, real-time and suitable for industrial applications.

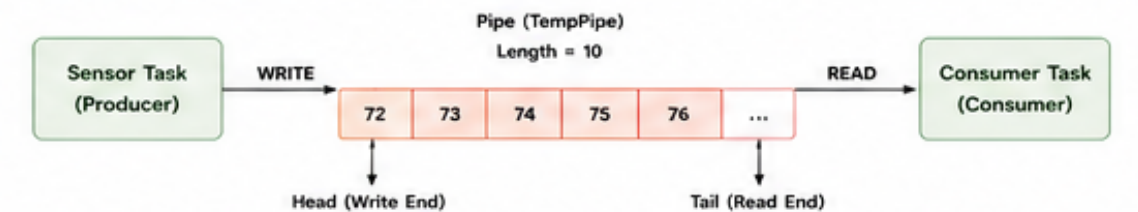
REAL-TIME BENEFIT IN INDUSTRY

In industrial plants like boilers, turbines, or compressors, sensors generate data periodically. Using a pipe ensures that data is transferred to the logging/monitoring system without loss and in the correct order. It improves system reliability, reduces CPU usage, and maintains real-time performance.

6. PIPE OPERATIONS SUMMARY

Operation	Used By	Function	Blocks When	Returns
WRITE_PIPE()	Sensor Task (Writer)	Writes data to pipe	Pipe is Full	PIPE_OK / PIPE_ERROR
READ_PIPE()	Consumer Task (Reader)	Reads data from pipe	Pipe is Empty	PIPE_OK / PIPE_ERROR
CREATE_PIPE()	Initialization	Creates a pipe	If insufficient memory	PIPE_OK / PIPE_ERROR
DELETE_PIPE()	System Shutdown	Deletes the pipe	-	PIPE_OK / PIPE_ERROR

7. EXAMPLE DATA TRANSFER



8. KEY POINTS

- Pipe is a unidirectional communication channel.
- Follows FIFO (First In First Out) order.
- Writer blocks when pipe is full.
- Reader blocks when pipe is empty.
- Ensures data integrity and synchronization.
- Prevents busy waiting and saves CPU time.
- Suitable for sensor data, logging, display, etc.

9. ASSUMPTIONS

- Pipe length is sufficient for normal operation.
- Data size (INTEGER) is fixed.
- Sensor reading time and consumer processing time are within real-time limits.
- RTOS supports pipe communication.
- No data corruption occurs in pipe.

10. GLOSSARY

Pipe	: A unidirectional communication channel with FIFO behavior.
FIFO	: First In First Out – data read in the same order as written.
Writer	: Task that writes data into the pipe.
Reader	: Task that reads data from the pipe.
Block	: Task waits if pipe is full (writer) or empty (reader).
Head	: Write position in the pipe.
Tail	: Read position in the pipe.

11. CONCLUSION

The pseudocode demonstrates how a sensor task writes data to a pipe and a consumer task reads it. Pipes ensure safe, ordered and efficient data transfer in RTOS-based industrial systems, making them ideal for real-time monitoring and control applications.